Contest FAQ

This FAQ summarises common questions raised during the contest and the corresponding guidance from the organising team.

1 Verdicts: RE, Crash, PE, and TLE

1.1 What does RE mean?

RE means Runtime Error. Your program crashed or stopped unexpectedly during evaluation.

1.2 Does RE mean Time Limit Exceeded?

No. RE and TLE are different. RE usually indicates that the program crashed, failed to follow the required input/output format, accessed unavailable files, or encountered an error during execution.

1.3 What does Crash mean?

If a submission receives Crash with 0 ms execution time, it usually indicates a contest-side technical issue. Such submissions may be re-tested automatically.

1.4 What is Presentation Error?

Presentation Error means the output format is incorrect. Please check that the required output file exists, has the correct name, valid JSON format, correct structure, and no extra unexpected output.

---

2 Common Runtime Errors and How to Avoid Them

Before submitting, please run:

```
Kernel → Restart & Run All
```

in JupyterLab. The evaluation system runs your notebook from a clean state, top to bottom.

2.1 Category 1: Python syntax and logic errors

2.1.1 Missing import or variable from an earlier cell

Symptom:

```
NameError: name 'torch' is not defined
NameError: name 'device' is not defined
```

This usually happens when a variable or import was created in an earlier cell that was not re-run.

2.1.2 Typo or wrong variable name

Symptom:

```
NameError: name 'txt' is not defined
```

For example, your code defines `words`, but later uses `txt`.

2.1.3 CUDA tensor converted to NumPy without `.cpu()`

Symptom:

```
TypeError: can't convert cuda:0 device type tensor to numpy
```

Incorrect:

```
mask = (a == b) & (c == d).detach().cpu().numpy()
```

Correct:

```
mask = ((a == b) & (c == d)).detach().cpu().numpy()
```

2.1.4 Writing to a read-only filesystem

Symptom:

```
OSError: [Errno 30] Read-only file system
```

Do not create or modify files inside `dataset/test_public/`. Write your output file to the working directory instead.

2.1.5 NumPy types are not JSON serializable

Symptom:

```
TypeError: Object of type int64 is not JSON serializable
```

Convert NumPy values to native Python types:

```
predictions = [int(x) for x in predictions]
```

2.2 Category 2: Reading labels from the test set

During scoring, `dataset/test_public/` is replaced by a hidden test set without `labels.json`. Only `dataset/train/` contains labels.

Note: different tasks may have different datasets and labels.

Symptom:

```
FileNotFoundError: dataset/test_public/labels.json
```

Incorrect:

```
labels_path = f"dataset/{self.split}/labels.json"
labels = json.load(open(labels_path))
```

Correct:

```
labels = json.load(open("dataset/train/labels.json"))

test_observations = json.load(open("dataset/test_public/observations.json"))
predictions = model.predict(test_observations)

with open("predictions.json", "w") as f:
    json.dump(predictions, f)
```

Do not attempt to read test labels during prediction.

2.3 Category 3: Hard-coded file paths

Avoid hard-coding dataset paths in a way that assumes the data will always be mounted exactly as it appears locally.

Recommended pattern:

```
import os
from pathlib import Path

DATASET_ROOT = Path(os.environ.get("DATASET_ROOT", "dataset"))
TRAIN_SPLIT = os.environ.get("TRAIN_SPLIT", "train")
TEST_SPLIT = os.environ.get("TEST_SPLIT", "test_public")

train_dir = DATASET_ROOT / TRAIN_SPLIT
test_dir = DATASET_ROOT / TEST_SPLIT
```

---

3 Opening a JupyterLab Terminal

Some troubleshooting steps require using a terminal.

To open a terminal in JupyterLab:

1. Click `File → New → Terminal`, or
2. Click the `+` launcher button and choose `Terminal`.

After opening the terminal, go to the relevant task folder before running commands:

```
cd ~/your_task_folder
```

Replace `your_task_folder` with the folder for the task you are working on.

---

4 Git, saving, committing, and submitting

4.1 The platform says “You’ve already submitted this solution.” What should I do?

Please save, commit, push, and refresh the submit popup.

Steps:

1. Save `solution.ipynb`.
2. Open the Git panel in JupyterLab.
3. Stage `solution.ipynb`.
4. Enter a commit message.
5. Commit.
6. Push.
7. Refresh the submit popup.
8. Submit the new commit.

4.2 My commit appears in JupyterLab but not on the contest page. Why?

You probably committed but did not push. Please make sure you click Push after committing.

4.3 Git says I am not in a repository. What should I do?

Make sure you are inside the task folder.

Open a JupyterLab Terminal as described in Opening a JupyterLab Terminal, then go to the relevant task folder:

```
cd ~/your_task_folder
```

For example, if your task folder is `potato`, use:

```
cd ~/potato
```

If the issue persists, raise a local technical support card.

4.4 Which file is submitted?

Only the notebook named `solution.ipynb` is used for submission.

4.5 How do I manage the versions of my notebook?

You may keep multiple versions of your notebook by copying files.

Suggested naming pattern:

```
cp solution.ipynb solution_v1.ipynb
cp solution.ipynb solution_v2.ipynb
cp solution.ipynb solution_experiment_xgb.ipynb
cp solution.ipynb solution_before_cleanup.ipynb
```

Use clear names that help you remember what changed.

Important: only the file named `solution.ipynb` is submitted. If you want to submit one of your saved versions, copy or rename that version back to `solution.ipynb` before committing and submitting.

Example:

```
cp solution_before_cleanup.ipynb solution.ipynb
```

Then open `solution.ipynb`, run `Kernel → Restart & Run All`, save, commit, push, refresh the submit popup, and submit.

---

5 Restoring the original notebook, baseline, or dataset files

The repository contains a Git branch named `original`, which stores the original files.

Before restoring anything, it is recommended to back up your current work.

5.1 How do I restore the original `solution.ipynb`?

Open a JupyterLab Terminal as described in Opening a JupyterLab Terminal, then go to the relevant task folder:

```
cd ~/your_task_folder
```

Back up your current notebook:

```
cp solution.ipynb solution_backup.ipynb
```

Restore the original notebook from the `original` branch:

```
git checkout original -- solution.ipynb
```

Then reopen `solution.ipynb` in JupyterLab.

5.2 How do I restore original dataset files?

Open a JupyterLab Terminal as described in Opening a JupyterLab Terminal, then go to the relevant task folder:

```
cd ~/your_task_folder
```

To restore the dataset folder from the `original` branch:

```
git checkout original -- dataset
```

If you only need to restore a specific file in the dataset, use:

```
git checkout original -- dataset/path/to/file
```

For example:

```
git checkout original -- dataset/train/observations.json
```

5.3 How do I restore other original files?

Use the same pattern:

```
git checkout original -- path/to/file_or_folder
```

Examples:

```
git checkout original -- local_test.py
git checkout original -- models
```

5.4 What if Git cannot restore the file?

If a required file is still missing or corrupted after restoration, raise a technical support card so the environment can be checked.

5.5 What should I do if `.ipynb_checkpoints` breaks the baseline?

Remove checkpoint folders:

```
find . -type d -name ".ipynb_checkpoints" -prune -exec rm -rf {} +
```

Then rerun the notebook.

---

6 Local testing and debugging

6.1 Where should I run local tests?

Run local tests from a JupyterLab Terminal, not from a notebook cell.

Open a JupyterLab Terminal as described in Opening a JupyterLab Terminal, then go to the relevant task folder:

```
cd ~/your_task_folder
```

6.2 How do I run a local test?

If the task provides a local test script, run it from the task folder.

For example:

```
python local_test.py solution.ipynb
```

If verbose output is supported:

```
python local_test.py solution.ipynb --verbose
```

The exact command may differ by task. Please follow the task statement or documentation.

6.3 Why does `local_test.py` say “No such file or directory”?

You are likely in the wrong folder, or the task may not provide a file named `local_test.py`.

First, make sure you are in the correct task folder:

```
cd ~/your_task_folder
```

Then check the files in the folder:

```
ls
```

6.4 Can I use `print()` for debugging?

Be careful with extra output. Some tasks require a strict output protocol, and extra stdout output may cause Presentation Error or Runtime Error.

If debugging output is needed, prefer stderr:

```
import sys
print("debug message", file=sys.stderr)
```

---

7 Libraries and models

7.1 Can I use installed libraries not listed in the statement?

You may use libraries that are installed in the environment and allowed by the task rules.

7.2 Can I install packages with pip?

No. Do not rely on internet downloads or installing packages during the contest.

7.3 Can I use CatBoost or XGBoost?

Yes.

7.4 Can I use TensorFlow?

No.

7.5 Can I modify files in the `models/` directory?

Do not rely on modified model files. During judging, model directories may be replaced by the original copies.

---

8 Scores, leaderboard, and final selection

8.1 Where can I find baseline scores?

Baseline scores are not directly provided. Submit the initial unmodified `solution.ipynb` to see the baseline score if you want to.

Note that you will use one submission for this.

8.2 Are scientific committee or jury scores shown?

No. Jury/scientific committee scores are not shown during the contest.

8.3 If I do not bookmark or select any submissions, which ones count?

Your two best submissions according to Leaderboard A will be selected automatically for final scoring.

8.4 If I select one submission, what happens to the second slot?

The second slot will be filled automatically with your next-best submission according to Leaderboard A score.

8.5 Will Leaderboard B run later?

Yes. Selected submissions will be run later for Leaderboard B.

---

9 Platform and technical issues

9.1 What should I do if Jupyter says “Server unavailable or unreachable”?

Try refreshing the page. If the issue persists, follow the technical issue process described in Technical issues and extra time requests.

9.2 What does IOPub data rate exceeded mean?

Your notebook tried to display too much output. Avoid printing full datasets or very large objects.

9.3 What should I do if translations are missing or unreadable?

Refresh with Ctrl+F5. If the issue persists, follow the technical issue process described in Technical issues and extra time requests.

9.4 Where is the documentation?

The documentation link is available on the desktop of your contest laptop.

9.5 How do I access the AI assistant?

Use the chat link on the desktop.

---

10 Routing of questions

Please route questions to the appropriate channel.

10.1 Yandex platform

Use the **Yandex platform** for:

- Task-related questions
- Issues or questions that technical support cannot handle

10.2 Tech support card

Raise a **Tech Support** card for technical or environment issues, including:

- Laptop issues
- Network issues
- Keyboard, mouse, or other peripheral issues
- Platform issues
- Jupyter Notebook issues
- Translation server issues
- AI chatbot issues

---

11 Technical issues and extra time requests

If you experience any technical issues during the competition, please follow the instructions below according to the time remaining.

11.1 From the start of the competition until one hour before the end

1. Raise the **Tech Support** sign and wait for a technical support staff member to assist you.
2. While waiting, please continue working on other problems. Do not stop working entirely.
3. The technical support staff will record the issue and the time affected. The Jury will use this information to determine your extra time automatically.
4. The Jury will notify you of your granted extra time through the Yandex platform approximately every two hours. You may need to refresh your browser to see the updated extra time.

Note: If extra time has been granted but is not visible on your screen, refresh your browser to update the displayed competition time.

11.2 From one hour to 30 minutes before the end

If the technical issue has still not been resolved, submit a request directly through the **Yandex platform** during this period:

- **Title:** Extra Time
- **Content:** State how much extra time you are requesting and explain the reason for your request.

Please note:

- Extra time requests may only be submitted during the 30-minute window from one hour to 30 minutes before the end of the competition.
- Approximately two hours before the end of the competition, the judging system may slow down due to heavy traffic. If your submission takes about one hour before it begins running, this is considered normal and will generally not qualify for extra time.

11.3 From 30 minutes before the end until the competition ends

If you still wish to submit an extra time request during this period, please remain at your seat:

1. Continue until the competition officially ends.
2. After the competition ends, do not leave your seat. Raise the **Tech Support** sign to show you need extra time.
3. Remain at your original competition seat and wait for the Jury to review your request.
4. Any extra time approved during this period will not exceed 30 minutes.

---

12 Submissions and judging

12.1 Why are my submissions pending for a long time?

There may be a long submission queue, especially near the contest deadline. A submission may remain pending even if your notebook runs quickly locally.

Pending time is not necessarily execution time; it may mean your submission is waiting for an available runner.

Near the deadline, the queue may take up to around 1 hour. Please plan submissions accordingly and avoid waiting until the final minutes where possible.

12.2 Does “pending” mean my code is currently running?

Not necessarily. In many cases, “pending” means the submission is still waiting in the queue and has not started execution.

12.3 Will submissions made before the contest ends still be evaluated after the contest ends?

Yes. Any submission made before the contest deadline will still be evaluated, even if it remains pending when the contest ends.

12.4 Can I select a pending submission for final scoring?

Yes. You may select/bookmark a pending submission. It will be evaluated later.

12.5 If I select only one submission, will the second one be selected automatically?

Yes. If you select only one submission, the other one will be selected automatically according to Leaderboard A score.

12.6 If I do not select any submissions, what happens?

Your two best submissions according to Leaderboard A will be selected automatically for final scoring.

12.7 Can I cancel a pending submission?

No. There is no way to cancel a submission once it has been submitted.

12.8 Can I submit another solution while one is pending?

Yes. You may continue working and submit another solution while a previous one is pending.

---

13 General contest logistics

13.1 Do failed submissions count toward the submission limit?

In general, submissions that produce RE, PE, or TLE count toward the submission limit. If a submission fails due to a confirmed contest-side Crash or infrastructure issue, the organisers may re-test it or handle it separately.

13.2 Can I continue editing while a submission is pending?

Yes. You may continue editing, committing, pushing, and submitting new versions while earlier submissions are pending.

13.3 Can I change the priority or cancel a pending submission?

No. Pending submissions cannot be cancelled or reprioritised.

13.4 Can I leave early?

Follow the contest staff's instructions. If you have an unresolved extra-time or technical issue request, remain at your seat.

13.5 What if I accidentally click Finish Contest?

Raise a technical support card immediately. In previously clarified cases, contestants were allowed to continue, but staff should confirm your specific situation.

13.6 Can I download files to my local machine?

No. Work should be done in the provided JupyterLab contest environment unless the organisers explicitly allow otherwise.

13.7 Do I need to screen record?

No. You do not need to screen record unless contest staff explicitly instruct you to do so.

13.8 Can I write intermediate files?

You may write files in your working directory if the task rules allow it, but do not rely on modifying dataset or model directories during judging.

13.9 Should my solution be deterministic?

Using a fixed random seed is recommended when randomness affects your output, so results are reproducible.

13.10 Can I check GPU usage?

You may check GPU usage from the JupyterLab terminal if standard tools are available, but do not modify system or environment settings.

---

14 Quick troubleshooting checklist

1. Run `Kernel → Restart & Run All` before submitting.
2. Make sure you are in the correct task folder.
3. Save, commit, push, and refresh the submit popup.
4. Submit only `solution.ipynb`.
5. If you manage versions by copying notebooks, rename or copy the version you want to submit back to `solution.ipynb`.
6. Do not include local testing commands inside the submitted notebook.
7. Avoid extra stdout output if the task requires a strict output protocol.
8. Use stderr for debugging output where appropriate.
9. Do not read `labels.json` or `answers.json` from the test split.
10. Use robust dataset paths or environment variables.
11. Do not write into `dataset/test_public/`.
12. Convert NumPy values to native Python types before writing JSON.
13. If dataset files are corrupted or missing, follow the technical issue process described in Technical issues and extra time requests.